

Makefile and justfile

- Research projects accumulate shell one-liners: **build figures, run tests, clean outputs.**

- Research projects accumulate shell one-liners: **build figures, run tests, clean outputs.**
- **Make** and **just** turn those steps into **named commands** you can run the same way every time.

Why use a task runner?

- Repeatability: `./fig1.pdf` depends on `analysis.py` and `data.csv`; the command that produces it lives in one place.

Why use a task runner?

- **Repeatability:** `./fig1.pdf` depends on `analysis.py` and `data.csv`; the command that produces it lives in one place.
- **Onboarding:** newcomers run `make all` or `just run` instead of hunting chat logs.

Why use a task runner?

- **Repeatability:** `./fig1.pdf` depends on `analysis.py` and `data.csv`; the command that produces it lives in one place.
- **Onboarding:** newcomers run `make all` or `just run` instead of hunting chat logs.
- **Documentation:** the `Makefile` or `justfile` is an index of “how this repo works.”

- `make` reads a file named **Makefile** in the current directory.

- `make` reads a file named **Makefile** in the current directory.
- A **target** is a label; **prerequisites** are inputs; the **recipe** is shell under the target.

- `make` reads a file named **Makefile** in the current directory.
- A **target** is a label; **prerequisites** are inputs; the **recipe** is shell under the target.
- Recipe lines start with a **tab character**, not spaces.

Makefile example

```
.PHONY: all run test clean
all: output/summary.csv figures/fig1.png
output/summary.csv: src/summarize.py data/raw.csv
    python src/summarize.py
figures/fig1.png: src/plot_figs.py output/summary.csv
    python src/plot_figs.py
test:
    python -m pytest tests/
clean:
    rm -f output/summary.csv figures/fig1.png
```

Makefile example

```
.PHONY: all run test clean
all: output/summary.csv figures/fig1.png
output/summary.csv: src/summarize.py data/raw.csv
    python src/summarize.py
figures/fig1.png: src/plot_figs.py output/summary.csv
    python src/plot_figs.py
test:
    python -m pytest tests/
clean:
    rm -f output/summary.csv figures/fig1.png
```

- `make all` builds `summary` then `fig1` (because the figure depends on the CSV).

Makefile example

```
.PHONY: all run test clean
all: output/summary.csv figures/fig1.png
output/summary.csv: src/summarize.py data/raw.csv
    python src/summarize.py
figures/fig1.png: src/plot_figs.py output/summary.csv
    python src/plot_figs.py
test:
    python -m pytest tests/
clean:
    rm -f output/summary.csv figures/fig1.png
```

- `make all` builds `summary` then `fig1` (because the figure depends on the CSV).
- change `data/raw.csv` → running `make all` refreshes what is out of date.

```
.PHONY: all hello clean
all: hello.txt
hello.txt:
    echo "Hello, COGS 205B" > hello.txt
hello:
    cat hello.txt
clean:
    rm -f hello.txt
```

```
.PHONY: all hello clean
all: hello.txt
hello.txt:
    echo "Hello, COGS 205B" > hello.txt
hello:
    cat hello.txt
clean:
    rm -f hello.txt
```

- make (or make all) creates hello.txt.

```
.PHONY: all hello clean
all: hello.txt
hello.txt:
    echo "Hello, COGS 205B" > hello.txt
hello:
    cat hello.txt
clean:
    rm -f hello.txt
```

- `make` (or `make all`) creates `hello.txt`.
- run `make` again with no changes: Make says it is already up to date.

- Sometimes a target does not naturally produce a useful output file, but you still want Make to know “this step already ran.”

- Sometimes a target does not naturally produce a useful output file, but you still want Make to know “this step already ran.”
- A **sentinel file** is a tiny marker file (often in `.sentinel/`) that records completion time for a step.

- Sometimes a target does not naturally produce a useful output file, but you still want Make to know “this step already ran.”
- A **sentinel file** is a tiny marker file (often in `.sentinel/`) that records completion time for a step.
- Downstream targets can depend on that marker instead of rerunning expensive setup.

Makefile example with sentinel files

```
.PHONY: all clean
all: report.txt
.sentinel/setup.done: requirements.txt
    mkdir -p .sentinel
    python -m pip install -r requirements.txt
    touch .sentinel/setup.done
report.txt: src/report.py data/input.csv .sentinel/setup.done
    python src/report.py --in data/input.csv --out report.txt
clean:
    rm -rf report.txt .sentinel
```

Makefile example with sentinel files

```
.PHONY: all clean
all: report.txt
.sentinel/setup.done: requirements.txt
    mkdir -p .sentinel
    python -m pip install -r requirements.txt
    touch .sentinel/setup.done
report.txt: src/report.py data/input.csv .sentinel/setup.done
    python src/report.py --in data/input.csv --out report.txt
clean:
    rm -rf report.txt .sentinel
```

- touch .sentinel/setup.done marks setup as complete.

Makefile example with sentinel files

```
.PHONY: all clean
all: report.txt
    .sentinel/setup.done: requirements.txt
        mkdir -p .sentinel
        python -m pip install -r requirements.txt
        touch .sentinel/setup.done
report.txt: src/report.py data/input.csv .sentinel/setup.done
    python src/report.py --in data/input.csv --out report.txt
clean:
    rm -rf report.txt .sentinel
```

- `touch .sentinel/setup.done` marks setup as complete.
- Edit `requirements.txt` and Make reruns setup before rebuilding `report.txt`.

1. Add a target whenever you repeat a command more than twice.

1. Add a target whenever you repeat a command more than twice.
2. Point **large artifacts** (CSVs, PDFs) at their **code + data** prerequisites so **make** skips useless work.

1. Add a target whenever you repeat a command more than twice.
2. Point **large artifacts** (CSVs, PDFs) at their **code + data** prerequisites so **make** skips useless work.
3. Keep **clean** destructive but **predictable**; use **.PHONY** for targets that are not real files (**test**, **all**).

- `just` runs recipes from a file called **justfile** (or `Justfile`).

- **just** runs recipes from a file called **justfile** (or **Justfile**).
- Syntax is closer to a **shell script with headings**: no built-in file timestamp rules like Make.

- **just** runs recipes from a file called **justfile** (or **Justfile**).
- Syntax is closer to a **shell script with headings**: no built-in file timestamp rules like Make.
- Install: package managers (**apt**, **brew**, **cargo install just**, releases).

```
# List recipes: `just --list`  
default:  
    @just --list  
run:  
    python src/summarize.py  
    python src/plot_figs.py  
test:  
    python -m pytest tests/  
clean:  
    rm -f output/summary.csv figures/fig1.png
```

```
# List recipes: `just --list`
```

```
default:
```

```
    @just --list
```

```
run:
```

```
    python src/summarize.py
```

```
    python src/plot_figs.py
```

```
test:
```

```
    python -m pytest tests/
```

```
clean:
```

```
    rm -f output/summary.csv figures/fig1.png
```

- `default` runs when you type `just` with no arguments; here it shows `just --list`.

1. Start every common action with `just <name>` so names stay short and memorable.

1. Start every common action with `just <name>` so names stay short and memorable.
2. Chain steps inside one recipe when order is **fixed**; split recipes when you run steps **alone** (`just test` only).

1. Start every common action with `just <name>` so names stay short and memorable.
2. Chain steps inside one recipe when order is **fixed**; split recipes when you run steps **alone** (`just test` only).
3. Prefer `just --list` (or a `default` that lists recipes) so the file **documents** itself.

1. Start every common action with `just <name>` so names stay short and memorable.
2. Chain steps inside one recipe when order is **fixed**; split recipes when you run steps **alone** (`just test` only).
3. Prefer `just --list` (or a `default` that lists recipes) so the file **documents** itself.
4. For “rebuild only when inputs change,” use **Make**; **just** focuses on **running commands**, not **change detection**.

Make vs just (when to reach for which)

	Make	just
Strength	Tracks file timestamps ; rebuilds only what changed	Readable recipes, friendly on boarding, cross-shell helpers
Friction	Tabs, quirky syntax, steep for big logic	No automatic dependencies between files; you encode order yourself
Typical use	Compilers, data pipelines with big intermediates	Project scripts: just dev , just lint , just slides

Make vs just (when to reach for which)

	Make	just
Strength	Tracks file timestamps ; rebuilds only what changed	Readable recipes, friendly on boarding, cross-shell helpers
Friction	Tabs, quirky syntax, steep for big logic	No automatic dependencies between files; you encode order yourself
Typical use	Compilers, data pipelines with big intermediates	Project scripts: just dev , just lint , just slides

Many teams use **both**: Make for heavy builds, **just** as a pretty front-end that calls **make** from a recipe.

- **Snakemake** extends Make-style dependency workflows for data science and bioinformatics.

- **Snakemake** extends Make-style dependency workflows for data science and bioinformatics.
- Like Make, rules declare **inputs**, **outputs**, and **commands**; unlike plain Make, it scales better to larger parameterized pipelines and cluster/cloud execution.

- **Snakemake** extends Make-style dependency workflows for data science and bioinformatics.
- Like Make, rules declare **inputs**, **outputs**, and **commands**; unlike plain Make, it scales better to larger parameterized pipelines and cluster/cloud execution.
- Keep the mental model: **dependency graph first**, command details second.

- GNU Make manual: www.gnu.org/software/make/manual

- GNU Make manual: www.gnu.org/software/make/manual
- **just**: github.com/casey/just (book in repo docs)

- GNU Make manual: www.gnu.org/software/make/manual
- **just**: github.com/casey/just (book in repo docs)
- Russell A. Poldrack, **Better Code, Better Science**: Workflows (automation and reproducibility)